Assignment 4 - Knowledge Graph Technical Report

Student Name: Guillermo Alfonso Medrano Sanchez / 高文昊
Student ID NTHU: 114065425
Student ID NCU: Z1140179

---

1. KG Construction from SQL with LLM

Building the Regulation and Article nodes is straight forward since the structure comes from SQLite. For processing the rules, several decisions were taken. A batch processing of articles was implemented to make the loading of the LLM more efficient. An important point in the design was shorter-lived connections to the container, since each processing of the articles may be lengthy — having one longer-lived session was unstable and may crash while building the rules.

- **Flow per Batch**

```
LLM inference → parse → fallback → validate → dedup → make_rule_id → embed
  └── THEN: session.open → write all rules (with embeddings) → session.close
```

The model used was **Qwen-2.5-3B**, run locally via a HuggingFace text-generation pipeline. Building the KG took a couple of hours.

Once rules are cleaned and deduplicated, each batch is passed through `embed_rules()` before being written to Neo4j. This function uses a small **all-MiniLM-L6-v2** (via `sentence-transformers`) to produce three 384-dimensional vector embeddings per rule: `action_embedding` encodes only the action clause, `result_embedding` encodes the consequence (skipped when the result is empty), and `combined_embedding` encodes the action and result concatenated. These vectors are stored directly on each Rule node and are indexed with cosine similarity vector indexes, enabling semantic similarity search alongside the full text indexes.

- **Summary of Profile given to LLM**

```
Goal: Extract structured rules from regulation article text.
```

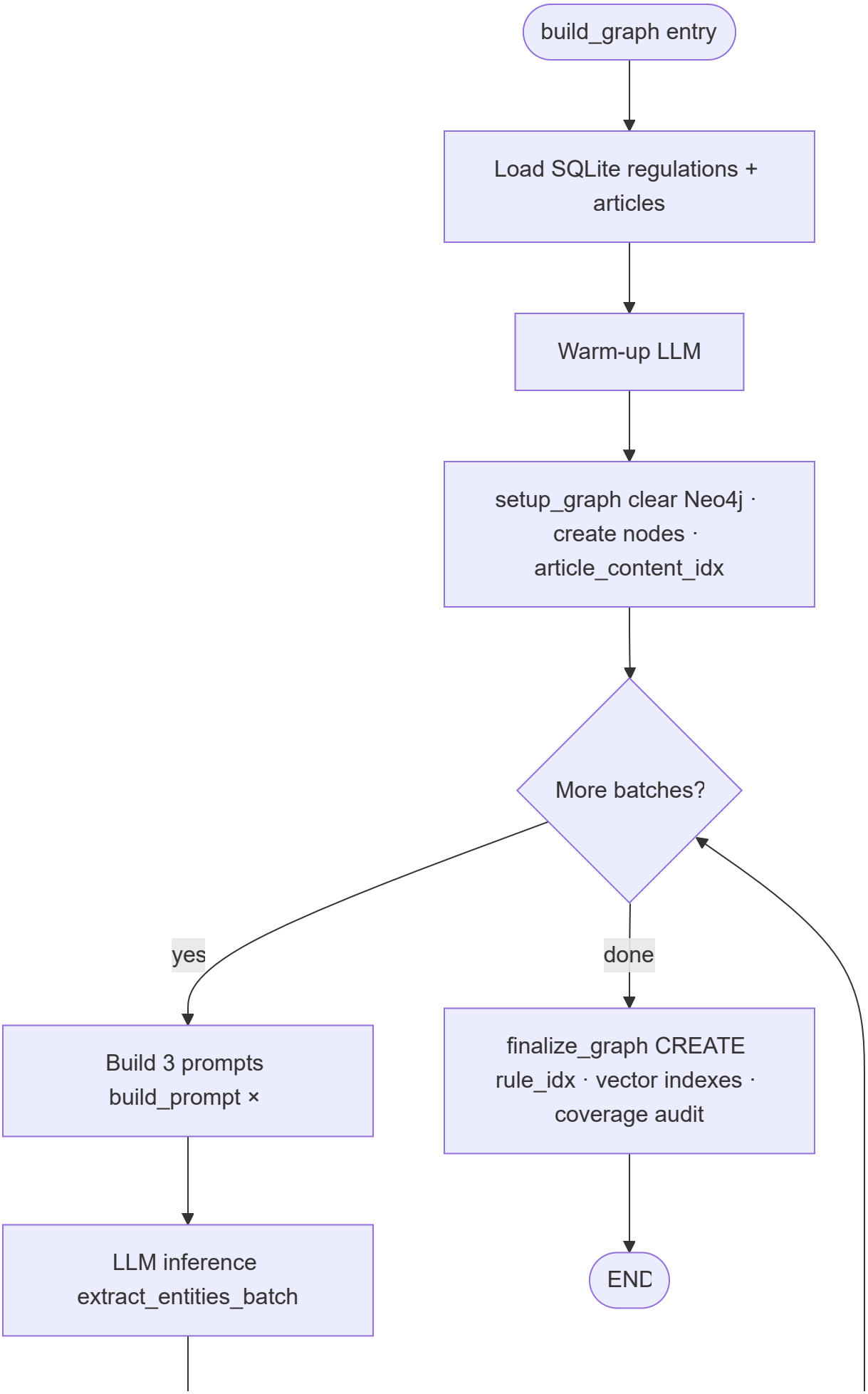
```
Output format: JSON only — a list of rule objects, each with `type`, `action`,
and `result`.

Six rule types the LLM must choose from exactly: `obligation`, `prohibition`,
`permission`, `penalty`, `procedure`, `other`.

Key constraints given to the model:
- Use verbatim quotes from the source text — no paraphrasing or summarizing
- One rule per distinct clause; a single article can produce multiple rules
- If a sentence carries both an obligation and its penalty, split into two
rules
- If `result`/consequence is absent, set it to `""`
- Skip purely administrative boilerplate with no clear rule
```

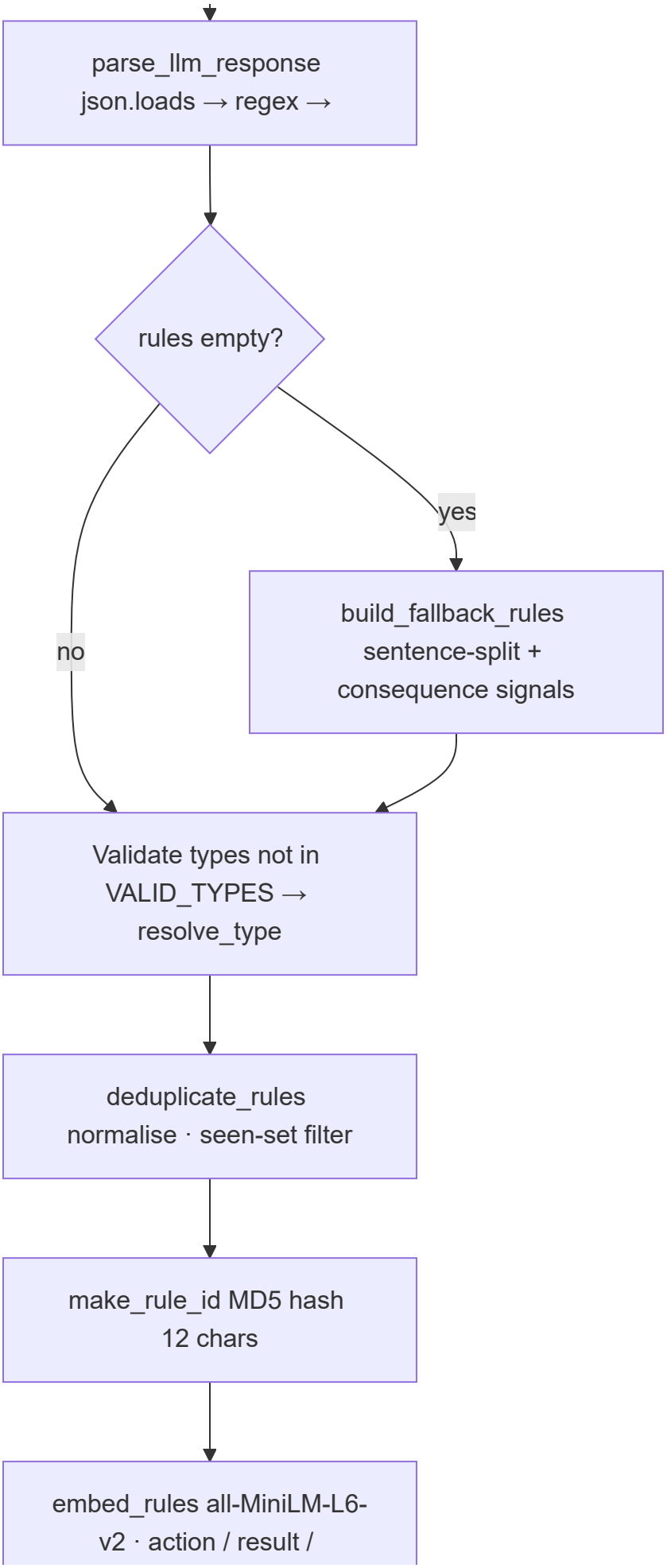
This is not the actual profile but a summary of it.

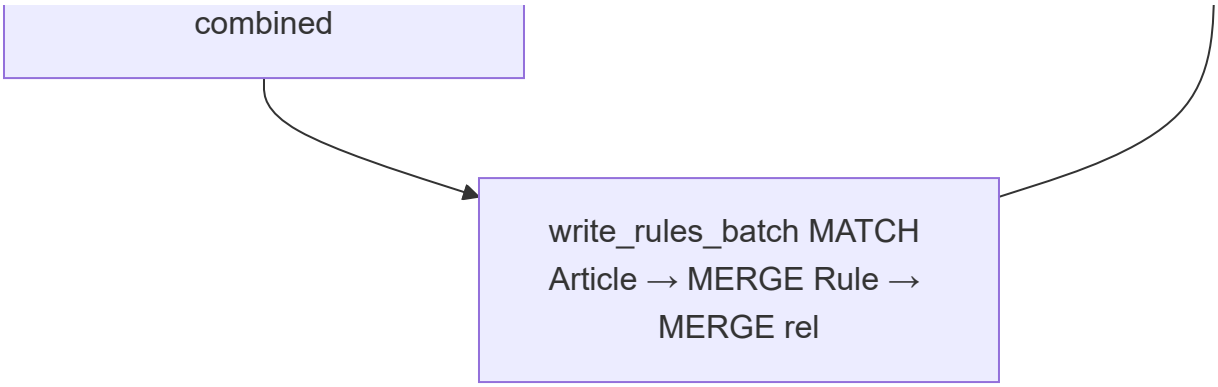
Generating several rules out of one article was requested in the profile, with instructions to stick to the wording of the original document — therefore it is easier to match words in future query retrievals. The LLM was given the profile to build several rules while making sure it kept the original words. This strategy generated a coverage of **158 out of 159 articles**.

The output of the LLM gets validated and then parsed to be able to be used to construct the Cypher instructions.

Mermaid Flowchart of KG Building Process


build_graph entry
Load SQLite regulations + articles
Warm-up LLM
setup_graph clear Neo4j · create nodes · article_content_idx
More batches?
yes
done
Build 3 prompts build_prompt ×
LLM inference extract_entities_batch
finalize_graph CREATE rule_idx · vector indexes · coverage audit
END


parse_llm_response
json.loads → regex →
rules empty?
yes
no
build_fallback_rules
sentence-split +
consequence signals
Validate types not in
VALID_TYPES →
resolve_type
deduplicate_rules
normalise · seen-set filter
make_rule_id MD5 hash
12 chars
embed_rules all-MiniLM-L6-
v2 · action / result /



---

2. Nodes

- **Node Properties**

Node	Property	Type / Notes	Source
Regulation	`id`	Integer — primary key	`regulations.reg_id`
	`name`	String	`regulations.name`
	`category`	String	`regulations.category`
Article	`number`	String — article identifier	`articles.article_number`
	`content`	String — full text (fulltext indexed)	`articles.content`
	`reg_name`	String — denormalised FK label	Joined from regulations
	`category`	String	Joined from regulations
Rule	`rule_id`	String — MD5 hash (12 chars)	`make_rule_id()`
	`type`	`obligation` | `prohibition` | `permission` | `penalty` | `procedure` | `other`	LLM or fallback
	`action`	String — verbatim clause from text	LLM extraction
	`result`	String — consequence (may be empty)	LLM extraction
	`art_ref`	String — article number	Batch metadata
	`reg_name`	String	Batch metadata

Node	Property	Type / Notes	Source
	`action_embedding`	List — 384-dim vector (all-MiniLM-L6-v2)	
	`result_embedding`	List — 384-dim vector; `null` when result is empty	
	`combined_embedding`	List — 384-dim vector of action + result concatenated	

- **Relationships**

Relationship	From	To	Cypher semantics
`:HAS_ARTICLE`	Regulation	Article	MERGE on reg id — idempotent re-runs
`:CONTAINS_RULE`	Article	Rule	MERGE on rule_id — idempotent re-runs

- **Indexes**

Index Name	Kind	Target Node	Indexed Properties
`article_content_idx`	Fulltext	Article	`content`
`rule_idx`	Fulltext	Rule	`action` , `result`
`rule_action_embedding_idx`	Vector	Rule	`action_embedding`
`rule_result_embedding_idx`	Vector	Rule	`result_embedding`
`rule_combined_embedding_idx`	Vector	Rule	`combined_embedding`

> All three vector indexes use **cosine similarity** with **384 dimensions** (matching `all-MiniLM-L6-v2` output). They require Neo4j 5.x or AuraDB.

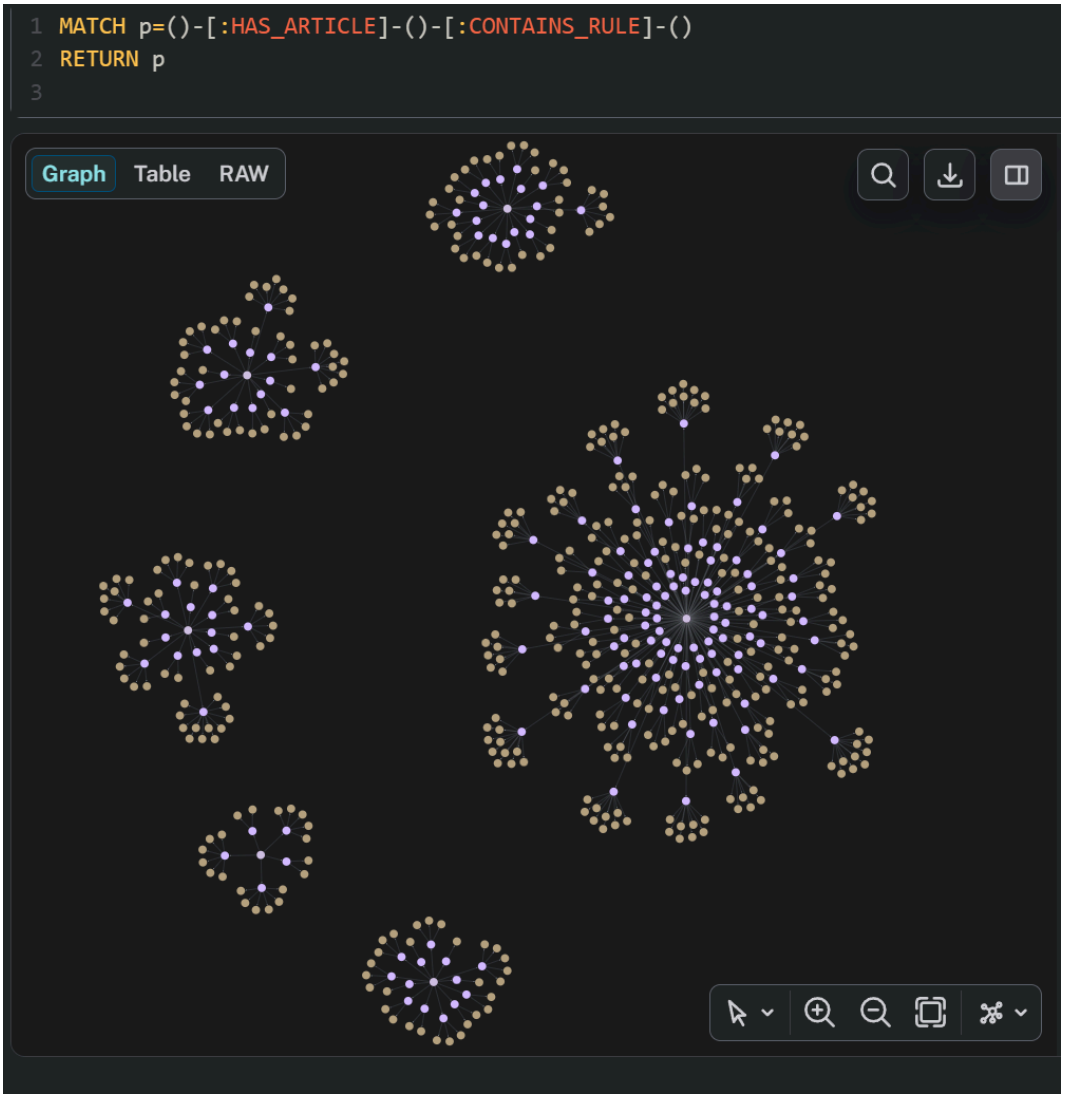
- **Schema Diagram**

```
┌──────────────────────┐
│      Regulation      │
│  ────────────────    │
│  id  · name          │
│  category            │
└──────────────────────┘
           │
           │  :HAS_ARTICLE
```

```
            ▼
 ┌──────────────────────┐
 │       Article        │
 │  ────────────────    │
 │  number · content    │
 │  reg_name · category │
 └──────────────────────┘
            │
            │  :CONTAINS_RULE
            ▼
 ┌──────────────────────────────┐
 │            Rule              │
 │  ──────────────────────────  │
 │  rule_id (MD5)               │
 │  type · action               │
 │  result · art_ref            │
 │  reg_name                    │
 │  action_embedding (384-dim)  │
 │  result_embedding (384-dim)  │
 │  combined_embedding(384-dim) │
 └──────────────────────────────┘
```

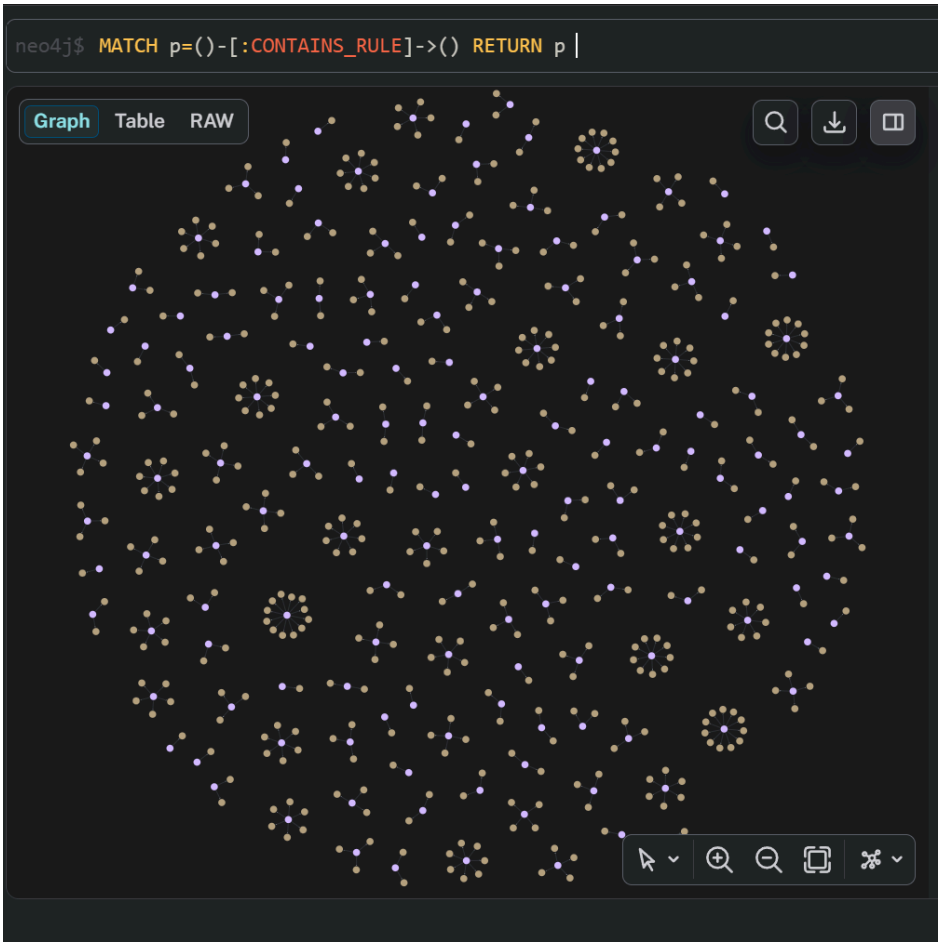
Nodes in Neo4j

Figure 1



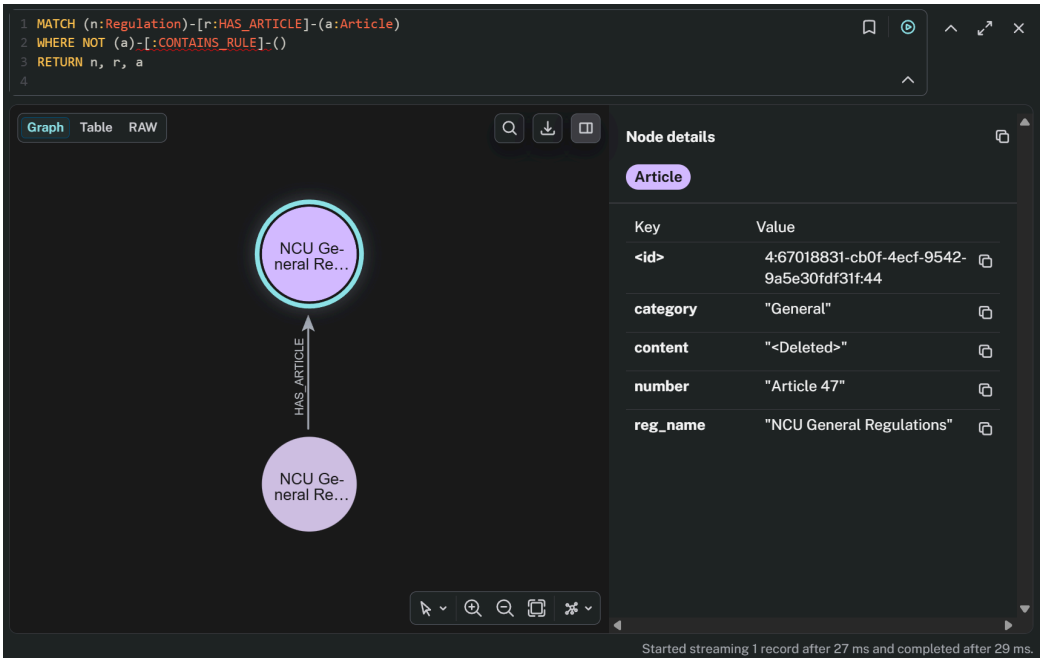
Full KG view of the 6 regulation with their articles and the articles with their rules.

Figure 2



The View of articles that have rules

Figure 3



View of the only Article not covered (Content was Deleted)

3. Fallback Strategies for KG Building

Five complementary layers fire in dependency order to handle malformed, empty, or invalid LLM output.

#	Strategy	Function	Trigger
1	Reg-Name Type Mapping	`reg_name_to_type()` / `resolve_type()`	Always — provides the default type
2	LLM Type Validation	`build_graph()` rule-cleaning loop	LLM returns type not in `VALID_TYPES`
3	Sentence-Split Rule Builder	`build_fallback_rules()`	LLM returns empty `rules` list
4	JSON Parse Fallback	`parse_llm_response()`	`json.loads()` raises exception
5	Deduplication Guard	`deduplicate_rules()`	Always — runs before every Neo4j write

Strategy 1: Reg-Name Type Mapping
Regex-matches the lowercased regulation name against an ordered keyword list. First match wins. Returns `'general'` if nothing matches.

Strategy 2: LLM Type Validation
If the LLM returns a type string not in `VALID_TYPES`, it is silently replaced by the reg-name-derived type from Strategy 1.

Strategy 3: Sentence-Split Rule Builder
Triggered when `rules == []`. Splits article content on sentence-terminating punctuation. Consequence sentences are detected by signal keywords and attached as the `result` of the preceding action sentence.

Strategy 4 : JSON Parse Fallback
If the model returns something that is not valid JSON, `parse_llm_response()` returns `{"rules": []}`, which then triggers **Strategy 3** (sentence-split fallback) since the rules list is empty.

Strategy 5 : Deduplication Guard
Normalises `(type, action, result)` to lowercase + collapsed whitespace and stores in a `seen` set. Skips rules with empty `action`.

---

4. Retrieval Strategy and Cypher Query

The goal is to:

- Retrieve **relevant regulation rules**
- Aggregate them into **article-level relevance**
- Generate answers strictly grounded in retrieved content

Bellow I will explain in detail what was made to fulfill the goals

4.1 Pre Processing

Tokenization & Cleaning
I thought that matching tokens as words would be better than exact matching of full sentences. Since the match must be exact a full sentence have less chance to match. I also cleaned it by doing lower cases and removing stop words from the tokens since they add no matching power because they will appear in a lot of places.

steps:

- Lowercasing
- Regex tokenization
- Stopword removal

Token Expansion
Since words need to match, I expanded the word, by expanding I mean also adding to the list words that are different but capture same lexical meaning and also domain aware synonyms. I used two means, WordNet and LLM Expansion. I expanded each token by 5 variations. Some tokens were not expanded, this tokens I labeled as low-priority like student, course and exam, since this words are already from the domain and they are very likely to appear in several places, so I considered it was not necessary to expand them.

Two-stage expansion

1. WordNet
   - Fast, deterministic
   - Captures lexical variations
2. LLM Expansion
   - Domain-aware (regulation language)
   - Constrained:
     - max = 5 expansions per token
     - single-word only

I limited the expansion due to

- Too many expansions → **query drift**
- Too few → **miss recall**

4.2 Rules Cypher Query Design & Retrieval Strategy

This system implements a **hybrid retrieval pipeline** combining:

- **Symbolic retrieval** (keyword voting + rule-based filtering)
- **Semantic retrieval** (embedding similarity)
- **Hierarchical aggregation** (rule → article)

Retrieval is a **two-stage Cypher pipeline**:
Rule-level candidate fetch **(** `build_voting_cypher` **)**
Scans all `:Rule` **nodes and applies a lightweight** whole-word regex pre-filter** on `action` and `result` fields. Only rules that match at least `MIN_BASE_TOKEN_MATCHES = 2` base tokens pass through. This avoids returning thousands of low-quality candidates while remaining tolerant of synonym mismatch (expansion tokens score later in Python, not in Cypher).

- **Minimum Match Constraint**
  Just 1 match will trigger too many rules and 3+ matches would make the matching sparce so I went with needing two matches. This may mitigate weak matches and promote semantic grounding.

MIN_BASE_TOKEN_MATCHES = 2

Applied in Cypher:
WHERE base_hit_count >= $min_base_matches

- **Rule Cypher retrieval**

```
MATCH (z:Rule)
WITH z,
  size([
    word IN $base_tokens                                -- (1)
    WHERE toLower(coalesce(z.action, '')) =~ ('.*\\b' + word + '\\b.*')
       OR toLower(coalesce(z.result, '')) =~ ('.*\\b' + word + '\\b.*')
  ]) AS base_hit_count
WHERE base_hit_count >= $min_base_matches             -- (2)
RETURN
    z.rule_id, z.type, z.action, z.result,
```

```
    z.art_ref, z.reg_name,
    base_hit_count,
    z.action_embedding,                                  -- (3)
    z.result_embedding,
    z.combined_embedding
```

#	Design decision	Reason
1	Whole-word regex `\b…\b` on `action` + `result` only	Keeps matching semantic fields; avoids false hits on IDs / metadata
2	`base_hit_count >= 2` gate	Eliminates single-token coincidences; keeps precision high before expensive Python scoring
3	Returns all three embeddings in one round-trip	Avoids a second DB call per rule; Python computes cosine similarity cheaply on the returned lists

- **Rule Retrieval (Neo4j)**

Quick overview
Rules are fetched by matches of tokens with the token threshold of at least 2 matches. The properties get fetch and once the properties in python, cosine similarity is done

Query returns per Rule:

- action text
- result text
- embeddings:
  - action_embedding
  - result_embedding
  - combined_embedding

4.3 Scoring of methods

- **Token Vote Score**

A token match score was computed, at them to get a final raw score for tokens.
Computed using:

- Whole-word regex matching
- Weighted token contributions

`votes += weight`

- **Token Weighting Strategy**

Since I have different set of words, I used a weighted strategy to try to put more weight into the words from the query and lower for the words that may trigger more matches like low priority ones (for example exam or student) and also expanded words which may also match not useful rules.

Weights:

Token Type	Weight
Base tokens	4
Low-priority	1
Expanded tokens	1

- **Embedding Score**

Once embeddings are in python, a cosine similarity is ran in the three embeddings. Then again I proceed to calculate an embedding score that used different weights.

score =
0.50 * combined_embedding

- 0.30 * action_embedding
- 0.20 * result_embedding

I considered that the combined embedding may capture better the request, that's why I gave it a higher weight.

- **Vote Normalization**

A normalization is done after, so the results are squashed into [0,1], is easier to compare and makes vote score in line with other scores.

norm_vote = vote_score / max_vote

- **Final Score**

A final hybrid score is computed which used both the normalized token vote and the embedding score.

final_score =
0.55 * norm_vote

- 0.45 * embedding_score

- **Rule Selection**
  Once scores are computed, the rules are sorted and the highest scoring rules are kept.

4.4 Article-Level Aggregation and Selection

Selection
Since one article can contain several rules and the question may trigger several of this rules that belong to the same article, I decided to aggregate at article level now.

For example
Article 3 had three distinct rules appearances in the top rules while Article 78 had 4, they appearances get aggregated and a new ranking is made according to this number, an article ranking. Basically we moving from rule level to article level.

Article Selection
The top 4 articles in the ranking are selected and a simple cypher query is produced to fetch the property of content of each of them. This way, now the LLM will get the full relevant article context which matched from rule level query.

Cypher Article Query fetch

```
MATCH (a:Article)
WHERE a.number IN $art_refs                -- top-4 from aggregation
RETURN
    a.number  AS art_ref,
    a.content AS content
```

4.5 Answer Generation

From grounding the LLM

the following profile , rules and instructions were given to the LLM

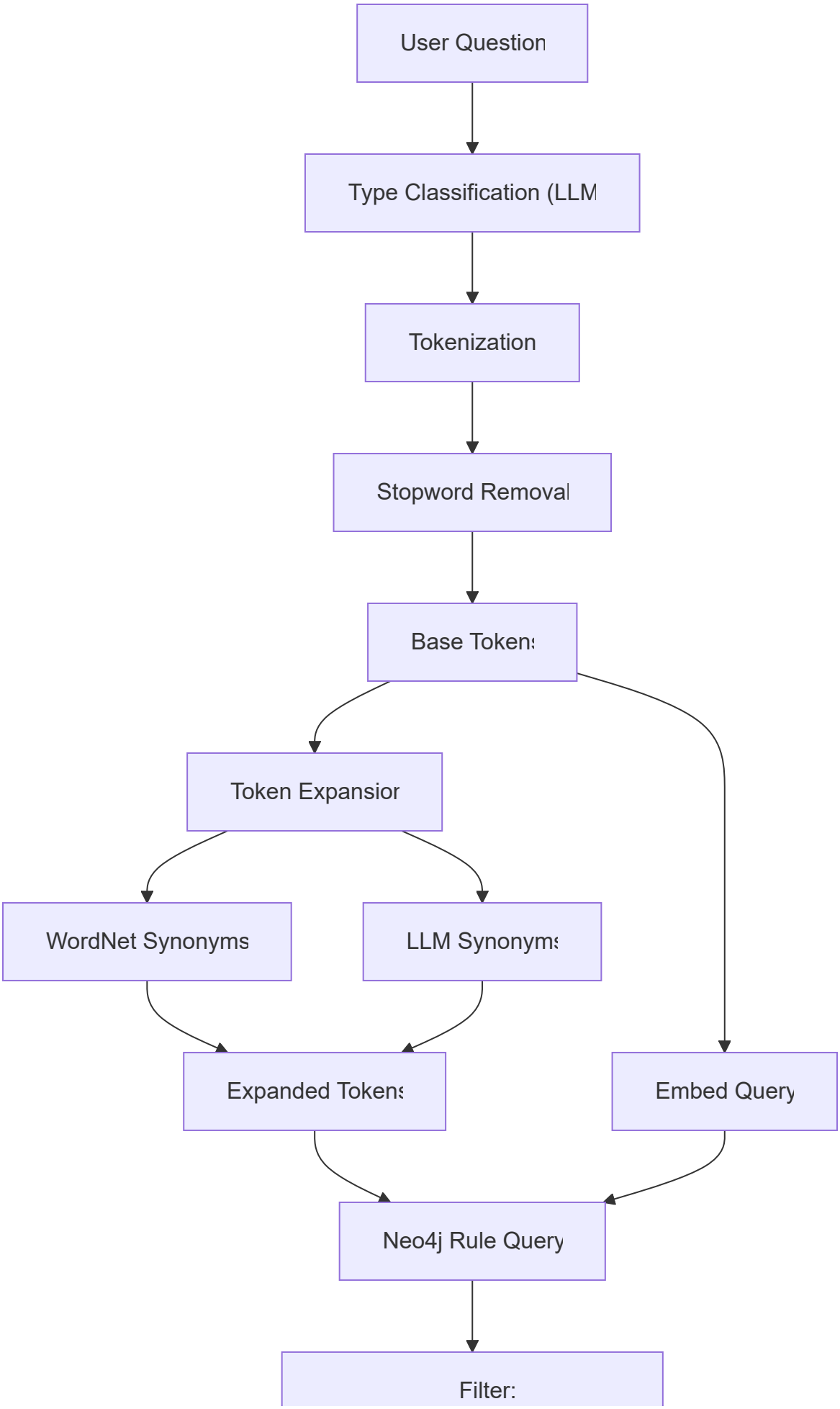
```
You are a careful regulation QA assistant.

STRICT RULES:
- Use ONLY the article content provided below. Do not use any outside
knowledge.
- You may ONLY cite these sources: {valid_refs_str}
- If you cannot find the answer in the provided articles, say: "The provided
```

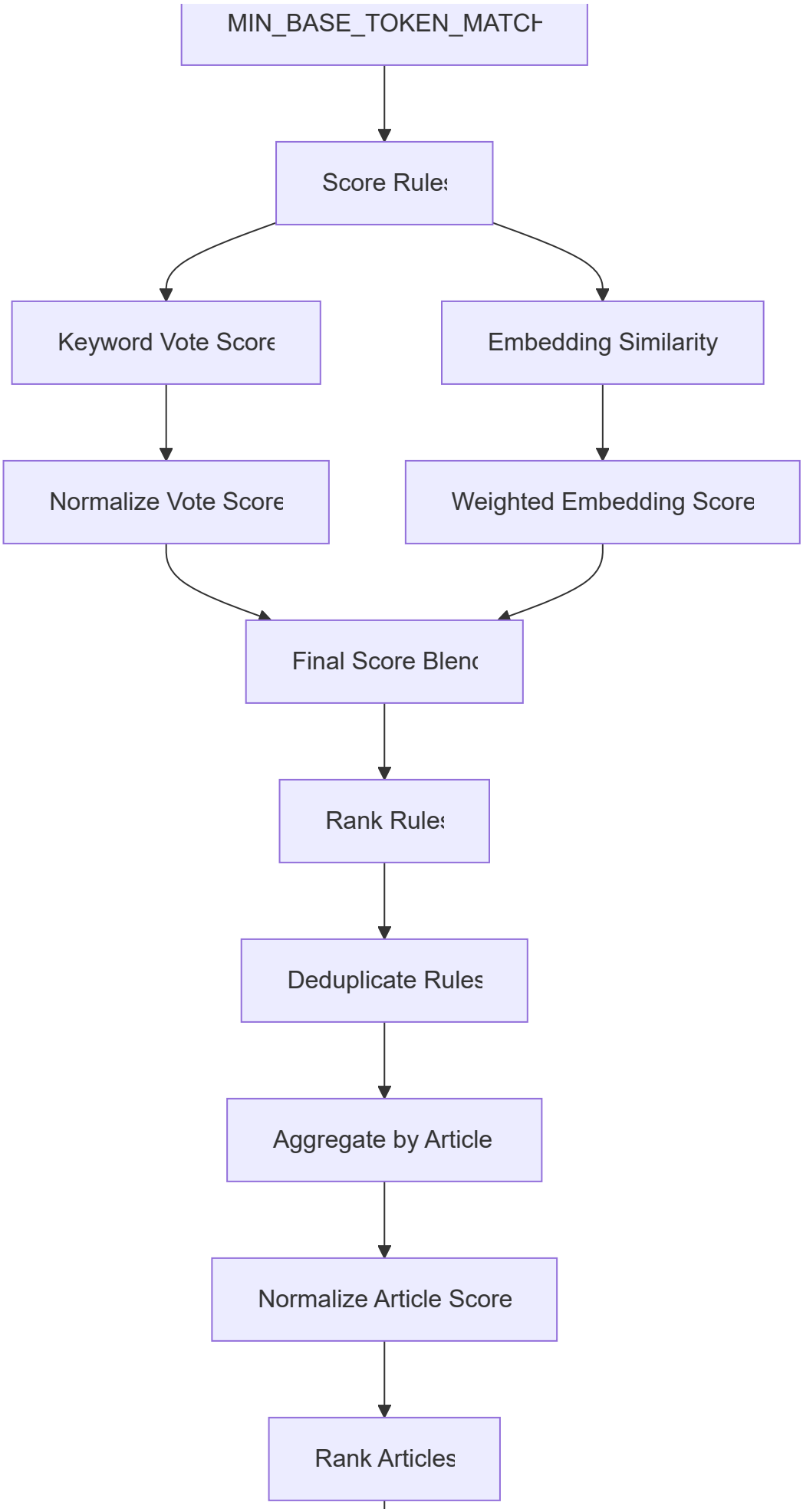
```
articles do not contain a clear answer to this question."
- NEVER invent facts, numbers, or article references not present in the
content below.
- A wrong answer with a fake citation is worse than admitting the answer is
not found.

Instructions:
- Cite the supporting source at the end: [Source: <art_ref>]
- Be concise and direct. Short answer, do not over explain.
```

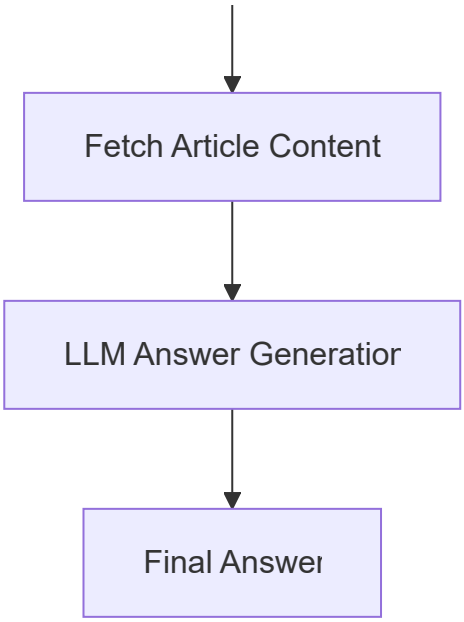
Important

The rules are forcing the agent to use sources for citation, forcing grounding. I define that it may not invent citations or facts, better for it to answer that the answer is not found. I also tell it to be concise and direct to avoid long explanations which may even lead to hallucination.

4.6 Control Flow Preprocessing + Retrieval + Response


User Question
Type Classification (LLM
Tokenization
Stopword Removal
Base Tokens
Token Expansion
WordNet Synonyms
LLM Synonyms
Expanded Tokens
Embed Query
Neo4j Rule Query
Filter:


MIN_BASE_TOKEN_MATCH
Score Rules
Keyword Vote Score
Embedding Similarity
Normalize Vote Score
Weighted Embedding Score
Final Score Blend
Rank Rules
Deduplicate Rules
Aggregate by Article
Normalize Article Score
Rank Articles



5. Fallback Strategies

The pipeline has 3 layered fallback mechanisms, listed from earliest to latest in the flow:

Strategy 1: classifier fallback
Checks if LLM returns valid type or if Json is malformed.

Strategy 2: Database disconnection guard
If driver is none or fails to connect, the system will print `"Unable to answer: database is not connected."`) immediately, preventing a crash cascade.

6. Failure analysis + improvements made

On my first run building the KG, it took several hours. I was doing sequential article feeding. I realized I could fed by batches to speed it up. It did speed it up but I notice that it still seemed slow, but it was due to Neo4J dropping in the middle due to the LLM taking too long to process. I was using one big connection the whole time, so decided to open connection to write and then close each time.

Also in the first runs, I had a rule that checked if result and action had text, this caused that a lot of valid rules were discarded due to having an empty result. By changing this to checking if actions had text, the rule nodes finally were populated.

My first approach was using a naive matching of summarized query from user with the rules but this returned sparse matching. I realized that direct matching was very unlikely and on top of that is a full sentence matching, so I started using tokens. I then decided to expand the tokens. At this point my accuracy was still not very good, so this is where I added weights and then decided to also use embeddings to be able to map the semantic meaning. Even if words are different, semantic meaning is likely to be similar.

The system has a fair amount of hyper parameters that could have been sweep with a hyperparameter optimizer to find a better combinations, I am sure some accuracy was left in the table by lacking this.

The matching with words was surprisingly harder than I thought, had to come with more strategies to be able to improve my results.